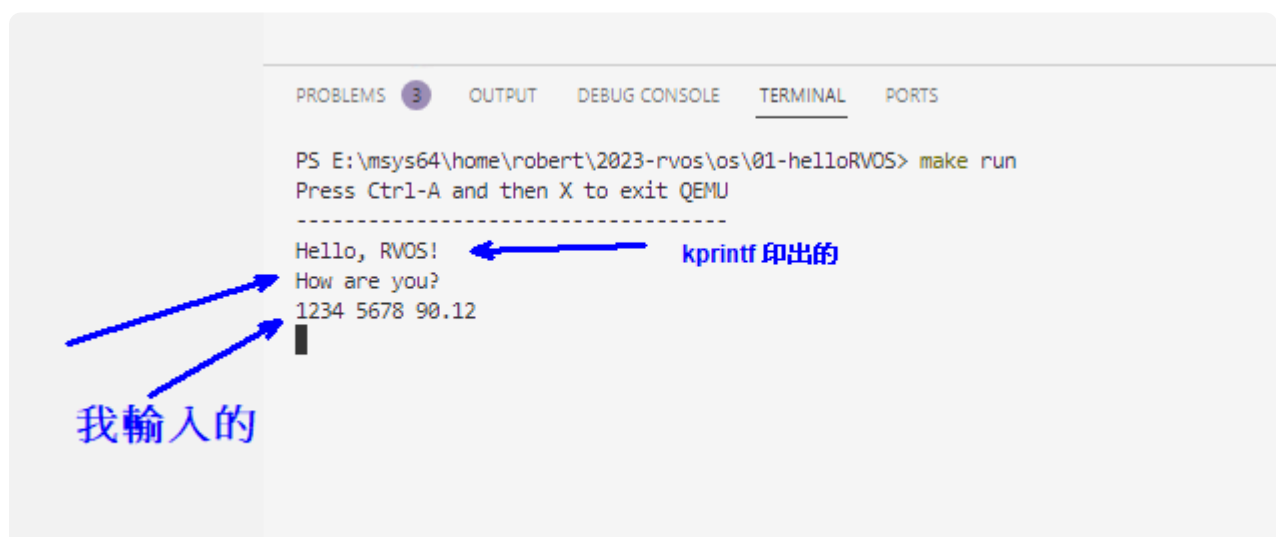


- 01-helloRVOS作業 (至少做一題，多做加分)
 - 作業一
 - 作業二
- 02-memory
 - 作業
- 03-task and 04-multitask (至少做一題，多做加分)
 - 題目一
 - 題目二
- 06-hwtimer

01-helloRVOS作業 (至少做一題，多做加分)

作業一

題目: 參考 code/os/01-helloRVOS，在此基礎上增加採用polling(輪詢)方式讀取控制台(PC過來)上輸入的字元並顯現在終端機上。另外當用戶按下Enter能夠另起一行從頭開始。



```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\msys64\home\robert\2023-rvos\os\01-helloRVOS> make run
Press Ctrl-A and then X to exit QEMU
-----
Hello, RVOS!
How are you?
1234 5678 90.12
█
```

我輸入的

kprintf 印出的

作業二

題目: 參考kprintf，寫一個 kscanf類似 scanf，可接收控制台(PC端)輸入格式化的資料，只需要接受 %d, %s

```
kscanf("input two integer %d, %d", &x, &y);  
kscanf("%s", &z);
```

02-memory

作業

題目: 參考 code/os/02-memanagement，在 page 分配的基礎上實踐更精細的動態記憶體管理，精確到以byte為單位。

要求實作以下函數介面，可參考 man(3) malloc：

```
void *malloc(size_t size);  
void free(void *ptr);
```

03-task and 04-multitask (至少做一題，多做加分)

題目一

題目: 04-multitask 中實現的任務調度中，前一個用戶任務直接呼叫 task_yield() 函數並最終呼叫

switch_to() 切換到下一個用戶任務。這樣的task_yield() 將會kernel借用了用戶任務的stack，當用戶任務的函數呼叫層次(函數裡又呼叫函數)過多或者 task_yield() 本身函數內部繼續調用函數，可能會導致用戶任務的stack空間溢出。參考為kernel調度單獨實現一個任務，在任務切換中，前一個用戶任務首先切換到kernel調度任務，然後再由kernel調度任務切換到下一個用戶任務，這樣就可以避免前面提到的問題了。參考以上設計方法，並嘗試實現之。

題目二

題目: 利用 RISC-V的 A(原子指令)實作 spin_lock 用以取代 04-multitask 的 lock, unlock

06-hwtimer

題目: 參考 06-hwtimer，利用系統提供的 TICK 機制實現一個數字時鐘，通過UART顯示在控制台上，隨著 TICK 的增加以牆上時鐘的效果顯示當前時間，顯示效果如下：
初始狀態：

```
Press Ctrl-A and then X to exit QEMU
-----
Hello, RVOS!
00:00:00
```

- 一秒後:

```
Press Ctrl-A and then X to exit QEMU
-----
Hello, RVOS!
00:00:01
```

...

- 一分鐘後:

```
Press Ctrl-A and then X to exit QEMU
-----
Hello, RVOS!
00:01:00
```

...